

Auswertung des Experteninterviews mit Prof. Mathias Goldau

1. Einstieg:

Frage:

Welche Rolle nehmen Sie derzeit in der Lehre von Programmierung bzw. objektorientierter Programmierung an der HTWK Leipzig ein?

Antwort:

- Professur für Angewandte Informatik mit Schwerpunkt Programmierung
- Aktiv seit 2016
- Betreuung über 1. bis 4. Semester von Modulen mit Programmierbezug, darunter Grundlagenveranstaltungen in C++ und Java sowie praxisorientierte Programmiermodule in höheren Semestern
- Beobachtung von Studierenden über verschiedene Ausbildungsphasen hinweg
- Einblick in die Vermittlung von Grundlagen als auch in die Anwendung der Programmierung in praxisnahen Projekten
- Mastermodul Human Computer Interaction, mit randweiser Programmierung (insbesondere Command Line Interfaces, etc.) und Fokus auf Psychologie (UX / UI Design)

Frage:

Welche Herausforderungen beobachten Sie bei der Vermittlung objektorientierter Programmierkonzepte besonders häufig?

Antwort:

- Ansammlung von vielschichtigen Problemen und Herausforderungen
- Heterogenität der Studierenden stellt zentrales Problem dar:
 - o Teilweise haben Studierende bereits umfangreiche Programmiererfahrungen aus Schule oder Freizeit erworben
 - o Andere Studierende haben kaum Vorkenntnisse (fehlende Informatikbildung in der Schule mit als Hauptgrund)
- Weitere Herausforderungen:
 - o Fehlende Grundlagenkenntnisse bei Studienbeginn

- Geringe Beschäftigung mit Programmierung außerhalb der Lehrveranstaltungen
- Begrenzte Zeitressourcen insbesondere bei dual Studierenden
- Große Leistungsunterschiede innerhalb einer Kohorte
- Entstehung der Notwendigkeit, grundlegende Konzepte teilweise erneut zu vermitteln, obwohl Studierende bereits deutlich fortgeschrittener sind

2. OOP-Lehre:

Frage:

Welche grundlegenden Konzepte der objektorientierten Programmierung halten Sie für besonders wichtig?

Antwort:

- Klassen & Objekte:
 - Unterschied zwischen beiden zählt zu den häufigsten Verständnisproblemen
 - Viele Studierende verwenden die Begriffe synonym oder können deren Beziehung nicht korrekt erklären
- Vererbung & Polymorphie:
 - Vererbung wird häufig verwendet, ohne dass die zugrundeliegenden Prinzipien verstanden werden
 - Polymorphie-Konzept bereitet Schwierigkeiten
- Interfaces & abstrakte Klassen:
 - Unterschiede zwischen Interfaces, abstrakten Klassen und konkreten Klassen werden oftmals nicht vollständig verstanden
- Datentypen & Kapselung:
 - Probleme beim Verständnis von primitiven Datentypen, Referenztypen und der Datenkapselung
 - Häufig keine Hinterfragung, weshalb Daten verborgen werden sollten
- Rekursion:
 - Wiederkehrendes Problemfeld, unabhängig vom verwendeten Paradigma
- Softwarestrukturierung:
 - Zufällige Verteilung von Programmcode auf Klassen („Spaghetti-Code“)
 - Schwierigkeiten der Studierenden Verantwortlichkeiten sinnvoll auf Klassen aufzuteilen

Frage:

Welche Lernmethoden funktionieren Ihrer Erfahrung nach besonders gut für die Vermittlung von OOP?

Antwort:

- Viele praktische Übungen (Übungsserien, Live-Programmierung)
- Visualisierungen und Tafelbilder
- Anschauliche Metaphern (RAM = lange Straße mit Adressen)
- Praxisnahe Prüfungsformen im Sinne des Constructive Alignment
- Wettbewerbe und spielerische Herausforderungen
- Kontinuierliche Beschäftigung mit Programmierproblemen

Frage:

Welche Rolle spielen praktische Übungen beim Erlernen von OOP-Konzepten?

Antwort:

- sehr wichtiges Element bei der Vermittlung

Frage:

Bitte bewerten Sie die Bedeutung von Visualisierung und Interaktivität in der OOP-Lehre auf einer Skala von 1 bis 5:

Antwort:

- 4 = eher wichtig
- Studierende sind zum großen Teil mit Gamification und Computerspielen aufgewachsen
- Praxisnahe und erlebbare Erfahrung mit den OOP-Konzepten

3. Lernschwierigkeiten der Studierenden:

Frage:

Bei welchen OOP-Konzepten treten bei Studierenden besonders häufig Verständnisprobleme auf? + Welche typischen Missverständnisse treten bei den Studierenden auf?

Antwort:

- Klassen & Objekte werden oft miteinander vermischt und verwechselt
- Unterscheidung von return und print ist nicht vorhanden
- Teilweise falsche Verwendung von Programmierkonstrukten (bspw. Operatoren)
- Missverständnisse entstehen durch mangelnde Auseinandersetzung mit den Programmierproblemen

4. Anforderungen an das Spiel „Deep Space Debugger“ für den Lernerfolg:

Frage:

Sehen Sie Potenzial in Serious Games zur Vermittlung objektorientierter Programmierung?

Antwort:

- Grundsätzlich ist Potenzial für den Einsatz von Serious Games in der Programmierausbildung vorhanden
- Genannte Vorteile:
 - o Hohe Vertraulichkeit der heutigen Studierenden mit digitalen Spielen
 - o Unterstützung visueller Lernprozesse
 - o Motivierende Lernumgebungen
 - o Möglichkeit zur Veranschaulichung abstrakter Konzepte

Frage:

Welche Grenzen oder Herausforderungen sehen Sie beim Einsatz von Serious Games in der OOP-Lehre?

Antwort:

- Klassische Lehre wird durch diese nicht gänzlich ersetzt werden können, sondern ergänzen diese
- Verbindung zwischen Spielinhalten und anschließender / dazugehöriger Lehrveranstaltung sind unabdingbar
- Motivation sollte möglichst intrinsisch erfolgen
- Keine reinen Belohnungssysteme oder starke Wettbewerbsorientierung, da sonst extrinsische Motivation im Vordergrund stehen kann

Frage:

Welche OOP-Konzepte sollten in einem Serious Game unbedingt behandelt werden?

Antwort:

- Klassen und Objekte
- Kommunikation zwischen Objekten
- Datenkapselung
- Vererbung
- Polymorphie
- Interfaces
- Komposition

- Typsicherheit
- Generics & Komplexität
- Als besonders geeignet erscheinen Konzepte, die sich visuell darstellen lassen und häufig zu Verständnisproblemen führen

Frage:

Welche OOP-Konzepte sind Ihrer Meinung nach zu komplex für ein Serious Game?

Antwort:

- Bezug Herstellung zu Typen ist schwierig zu realisieren
- Funktionale Programmierung darf sich nicht mit der objektorientierten Programmierung vermischen
- Funktionskapselung / Single Point of Concern
- Integration von UML / Klassendiagrammen eventuell zu komplex

Frage:

Welche didaktischen, spielerischen oder technischen Anforderungen müsste ein Serious Game erfüllen, damit tatsächlich ein Lernerfolg entstehen kann?

Antwort:

- leichter Zugang (idealerweise Browser-Anwendung)
- hohe technische Stabilität (ausführliches Debugging, Tests)
- Verwendung fachlich korrekter Begrifflichkeiten
- multimediale Aufbereitung (Bild, Sound, usw.)
- Anpassbarkeit der Inhalte / Editierbarkeit der einzelnen Level
- geringe Wettbewerbsorientierung

Ableitungen für „Deep Space Debugger“

Interviewergebnis	Konsequenz für das Spiel
Klassen und Objekte werden häufig verwechselt	Eigenes Level zu Klassen und Objekten
Polymorphie wird schlecht verstanden	Zentrale Spielmechanik zu Polymorphie
Kommunikation zwischen Objekten wichtig	NPCs senden Nachrichten / Methodenaufrufe
Visualisierung hilfreich	Objektbeziehungen sichtbar machen
Viele Übungen notwendig	Mehrere kleine Aufgaben statt langer Erklärtexpte
Wettbewerb nur begrenzt sinnvoll	Fokus auf Problemlösen statt Highscores
Browser-Zugänglichkeit wünschenswert	Web-Export als Veröffentlichungsoption

